PhenoCam Phenology Project

Dataset, target and notebook walkthrough

Source: sudheer_code.ipynb. This guide explains the uploaded notebook and its saved outputs. The results describe that downloaded snapshot; the live CSV may change. The notebook contains four executed code cells, four heading cells and one empty code cell.

1. What is the project about?

The project studies how vegetation changes through the seasons. Camera colour measurements describe how green the vegetation is on different dates. The supplied starter approach predicts a daily greenness value and then derives seasonal transition dates from the predicted curve.

For the Student 1 direction, the intended comparison is between a 1D-CNN, a Bi-LSTM and a Temporal Transformer, with a Random Forest baseline. The wider project document focuses on the beginning and end of autumn senescence. These autumn dates describe the onset and completion of seasonal decline, rather than spring green-up.

Current progress: the notebook loads and inspects data only. It does not yet train a model or calculate seasonal dates.

Dataset used

The source is the Harvard site daily PhenoCam CSV specified by the supervisor. It contains numerical summaries extracted from camera images for a vegetation region of interest (ROI). The notebook works with the CSV measurements, not a folder of images.

Original CSV: harvard_DB_1000_1day.csv

Property	Saved notebook result
Rows / columns	6,779 daily records / 32 columns
Date coverage	4 April 2008 to 25 October 2026
Years	2008-2026; first and last years are partial
Valid greenness observations	6,661 non-missing gcc_90 values

2. Inputs, target and outputs

Immediate target: gcc_90. This is the daily 90th percentile of green chromatic coordinate (GCC) measurements. A percentile summarises the upper part of the daily greenness distribution; it is not a probability or a tumour class.

At the image-measurement level, GCC = G / (R + G + B). The daily gcc_90 target summarises multiple measurements. It is not generally identical to dividing the daily mean green value by the sum of daily mean red, green and blue values.

Column	Meaning and role
date	Observation date. Used for ordering, time features and splitting.
doy	Day of year (1-365 or 366). Selected model input.
r_mean	Daily mean red-channel value. Selected input.
g_mean	Daily mean green-channel value. Selected input.
b_mean	Daily mean blue-channel value. Selected input.
rcc_mean	Daily mean red chromatic coordinate. Selected input.
gcc_90	Daily 90th-percentile GCC. Regression target.
year / month	Calendar information for grouping and splitting; month has not yet been created.
image_count	Number of contributing images; available for quality checks.

Greenness prediction versus seasonal dates

The model first predicts a continuous greenness value. A later procedure identifies dates on the seasonal curve. Autumn start, autumn end and season length are project-level outputs, not the target columns currently selected in this notebook.

The same-day colour inputs are closely related to the greenness target. This setup evaluates reconstruction of greenness from observations. It should not be presented as forecasting future seasons unless the inputs, timing and evaluation are redesigned for forecasting.

Other columns

The CSV also includes midday image information, colour standard deviations, other GCC/RCC percentiles, maximum solar elevation and snow/outlier flags. They are loaded but not selected as model inputs. All five flag columns are empty in the saved snapshot, so they do not supply usable quality flags here.

3. Cell-by-cell explanation: imports

Notebook cell 1: the Markdown heading "Import libraries" labels the section. It does not execute Python.

Notebook cell 2: the first executable cell imports the tools and sets display preferences.

```
# Libraries for downloading and reading the dataset
import io
import requests

# Libraries for data handling
import numpy as np
import pandas as pd

# Libraries for plots in later sections
import matplotlib.pyplot as plt
import seaborn as sns

# Display settings
pd.set_option("display.max_columns", None)
sns.set_theme(style="whitegrid")

print("Libraries imported successfully.")
```

Code element	What it does
io	Lets pandas read downloaded CSV text as an in-memory file.
requests	Downloads the CSV from its URL.
numpy as np	Provides numerical operations for later processing; not yet used in these cells.
pandas as pd	Stores the data in a DataFrame and performs inspection.
matplotlib / seaborn	Prepare plotting tools for later analysis; no plots exist yet.
display.max_columns	Displays all columns when previewing a wide DataFrame.
sns.set_theme	Sets the appearance of future plots.
print(...)	Confirms that this cell reached its final line.

Saved result

The output is "Libraries imported successfully." This means the imports completed in the saved notebook run. It does not mean preprocessing or modelling has been performed.

4. Cell-by-cell explanation: data loading

Notebook cell 3: the Markdown heading introduces dataset loading. **Cell 4:** downloads and reads the dataset, then checks required columns.

```
# Original dataset URL from config.yml
DATA_URL = (
    "https://phenocam.nau.edu/data/archive/"
    "harvard/ROI/harvard_DB_1000_1day.csv"
)

# Download the CSV file
response = requests.get(DATA_URL, timeout=60)

# Stop and show an error if the download fails
response.raise_for_status()

# Remove metadata comments and blank lines
csv_text = "\n".join(
    line
    for line in response.text.splitlines()
    if line.strip() and not line.lstrip().startswith("#")
)

# Read the data into a DataFrame
df_raw = pd.read_csv(io.StringIO(csv_text))

# Remove accidental spaces from column names
df_raw.columns = df_raw.columns.str.strip()

# Check that the columns needed by the starter code are present
required_columns = [
    "date", "r_mean", "g_mean",
    "b_mean", "rcc_mean", "gcc_90"
]

missing_columns = [
    column for column in required_columns
    if column not in df_raw.columns
]

if missing_columns:
    raise ValueError(
        f"Required columns are missing: {missing_columns}. "
        "Check that the URL returned the correct CSV."
    )

print("Dataset loaded successfully.")
print("Number of rows:", df_raw.shape[0])
print("Number of columns:", df_raw.shape[1])

display(df_raw.head())
```

Meaning: requests.get downloads the file with a 60-second timeout. raise_for_status stops on an HTTP error. The text filter removes blank lines and metadata lines beginning with #. StringIO passes the remaining CSV text to pandas. Column names are stripped of surrounding spaces.

The missing_columns check verifies that all six essential source columns exist. If any are absent, the code raises a clear error. head() displays the first five rows; shape reports dimensions.

Saved result: 6,779 rows and 32 columns, with a preview beginning on 4 April 2008. This confirms that the expected columns were present in that run. It does not check all observation values or guarantee data quality.

5. Cell-by-cell explanation: inspection

Notebook cell 5: the heading introduces inspection. **Cell 6:** checks structure, missing data, duplicate rows and temporal coverage.

```
# Show all column names
print("COLUMN NAMES")
print(df_raw.columns.tolist())

# Show data types and non-missing counts
print("\nDATASET INFORMATION")
df_raw.info()

# Count ordinary missing values
print("\nMISSING VALUES")
print(df_raw.isna().sum())

# Count the supervisor's missing-value marker
# Check both numeric and text representations
marker_counts = df_raw.apply(
    lambda column: column.astype(str).str.strip().isin(
        ["-9999", "-9999.0"]
    ).sum()
)

print("\nMISSING-VALUE MARKERS (-9999)")
print(marker_counts[marker_counts > 0])

# Count completely duplicated rows
print("\nDuplicate rows:", df_raw.duplicated().sum())

# Inspect dates without changing the original data
dates = pd.to_datetime(df_raw["date"], errors="coerce")

print("\nDATE COVERAGE")
print("First date:", dates.min())
print("Last date:", dates.max())
print("Invalid dates:", dates.isna().sum())

print("\nNUMBER OF OBSERVATIONS PER YEAR")
print(dates.dt.year.value_counts().sort_index())
```

How to read it: columns.tolist prints names. info reports types and non-null counts. isna().sum counts missing cells in each column. marker_counts searches for the configured -9999 sentinel, including its decimal text form. duplicated checks identical whole rows.

to_datetime creates a temporary dates series; errors="coerce" turns invalid dates into NaT. The cell does not change df_raw["date"]. min/max report coverage, and year.value_counts reports the number of records per year. Duplicate dates and gaps between dates are not explicitly tested here.

6. What the inspection results mean

Check	Observed result and interpretation
Data types	27 float columns, 3 integer columns and 2 object columns. date is still stored as text.
Missing measurements	Each selected colour input and gcc_90 has 118 missing values (about 1.74% of 6,779 rows).
Empty flags	snow_flag and four outlierflag columns each have 6,779 missing values.
Missing-value sentinel	No -9999 or -9999.0 markers were found by this check. Ordinary NaN values still exist.
Whole-row duplicates	Zero completely identical rows. This does not separately establish unique dates.
Invalid dates	Zero dates failed conversion in the temporary check.
Calendar coverage	2008 has 272 records; 2026 has 298. Intermediate years have 365 or 366 records.

Missing-data interpretation

The matching non-null counts suggest that several measurement fields are missing on the same days, but the inspection does not explicitly verify their overlap. Therefore, the exact number of rows remaining after joint filtering has not yet been calculated. Do not report a cleaned row count as an executed result.

Preprocessing completed so far

The notebook removes metadata comments and blank lines from the downloaded text, strips spaces from column names, validates required columns, and preserves a raw copy. It also inspects missing data and dates. These are loading and preparation checks.

Preprocessing not yet completed

It has not replaced or imputed missing measurements, removed rows or columns, converted the working date column, sorted the working data, checked duplicate dates, created sequence masks, scaled inputs or split by year. No label encoding or SMOTE is required for this continuous greenness regression target.

No model results yet

There are no training results, RMSE/R-squared values, autumn dates, confusion matrices or model comparisons in this notebook. The saved outputs are dataset inspection results only.

7. Cell-by-cell explanation: model settings

Notebook cell 7: the heading introduces input and target selection. **Cell 8:** stores these settings and makes a working copy.

```
# Settings taken from the supervisor's configuration
INPUT_FEATURES = [
    "doy",       # Day of the year; we will create it from date
    "r_mean",    # Mean red-channel value
    "g_mean",    # Mean green-channel value
    "b_mean",    # Mean blue-channel value
    "rcc_mean"   # Mean red chromatic coordinate
]

TARGET = "gcc_90"  # 90th percentile green chromatic coordinate

# Keep the loaded data unchanged for reference
# We will clean this working copy in Part 2
df = df_raw.copy()

print("Input features:", INPUT_FEATURES)
print("Prediction target:", TARGET)
```

INPUT_FEATURES is a list of five column names, not a prepared training array. **TARGET** stores the target name. **df_raw.copy()** creates df so later edits can preserve the original DataFrame. No fitting, scaling or prediction takes place here.

Saved output: the five selected input names and target gcc_90 are printed. The source already contains doy; recreating it from a converted date column later will help keep date-derived fields consistent.

Notebook cell 9: an empty code cell. It contains no processing and no result.

Overall status

The notebook is a successful starting point for loading and inspecting the supplied dataset. The next step is basic preprocessing, followed by exploratory plots and an agreed temporal evaluation design.

8. What we will do next

Stage	Planned work - not yet executed
1. Basic cleaning	Convert date and numerical fields; replace sentinel/infinite values where present; check duplicate dates; sort chronologically. Keep an audit of changes.
2. Missing observations	Inspect missing dates and measurement gaps. Keep the calendar grid and missingness masks. Do not silently join observations across gaps or fill missing targets and treat them as true labels.
3. Explore seasonality	Plot gcc_90 across time and by year, observation availability and selected colour measurements. Examine autumn coverage.
4. Temporal split	Reserve earlier years for training, subsequent years for validation and later complete years for final testing. Select boundaries after quality checks; avoid random daily splitting.
5. Scale and build sequences	Fit a scaler on training inputs only. Build windows within allowed partitions and account for actual time gaps. Preserve target dates and masks.
6. Baseline and models	Train Random Forest, then 1D-CNN, Bi-LSTM and Temporal Transformer with comparable inputs and evaluation data.
7. Autumn transitions	Define the start/end rules and obtain reference transition dates. Apply the same rules consistently to held-out seasonal curves.
8. Evaluate and package	Report greenness error separately from date error in days; compare model size, training and inference times. Organise working code into the requested Python modules.

Decisions to settle before final experiments

The reference dates and autumn definitions are still unresolved. The supplied starter settings use a spring threshold and must be adapted. Camera greenness data at Harvard also differ from the wider document's Richmond Park satellite/terrain setting; the intended final scope should be agreed with the supervisor.

A single site supplies relatively few independent annual autumn events despite thousands of daily rows. Ten evaluation folds must respect time and contain enough seasonal events; overlapping daily windows must not be treated as independent annual validations. A multi-site extension may be needed for a stronger benchmark.

Simple explanation to a supervisor: I have loaded the Harvard PhenoCam daily measurements, checked their structure and missing values, and selected gcc_90 as the greenness target. I will next clean the data, prepare time-aware sequences and compare model architectures on held-out years. Seasonal date accuracy will be evaluated separately once reference autumn dates are established.